

Compiler Construction

Übung T-2 - LL(1)

Aufgabe 01 - Eigenschaften einer Grammatik

Besitzen die folgenden Grammatiken die LL(1)-Eigenschaft?

Begründen Sie Ihre Antwort in textueller Form (warum LL(1), warum nicht LL(1)). Sollte es mindestens eine LL(1)-Verletzungen geben, geben Sie bitte **alle** an. Geben Sie zudem für jede Grammatik, welche die LL(1)-Eigenschaft nicht besitzt, die theoretisch minimale Anzahl an Vorgriffssymbolen an, die für das Parsen der jeweiligen Grammatik notwendig wäre

a.)

```
Temperature = ["-"] ( Celsius | Kelvin ) .  
Celsius = { number } "°C" .  
Kelvin = { number } "K" .
```

Beispielsätze: 5°C, -12K, 273°C, 1000K

Nicht LL(1)-konform, da das First-Set von Celsius und Kelvin beides `number` enthält und die Mengen somit nicht disjunkt sein können. Man bräuchte mindestens 2 Lookaheads damit ein richtiges Parsing ermöglicht wird.

b.)

```
Locals = "class" ident "{" { VarDecl | FuncDecl } "}" .  
VarDecl = "int" ident { "," ident } ";" .  
FuncDecl = "fn" "int" ident "(" ")" ";" .
```

Beispielsatz: `class C { int a, b, c; fn int mul(); fn int add(); }`

LL(1) konform, da $\text{First}(\text{VarDecl}) = \text{"int"}$ und $\text{First}(\text{FuncDecl}) = \text{"fn"}$ sind. Somit sind beide Mengen disjunkt.

c.)

```
Var = ident ":" Type .  
Type = ( Primitive | FuncType ) .  
Primitive = "i32" | "f64" .  
FuncType = Type "->" Primitive .
```

Beispielsatz: `i: i32, pi: f64, id: i32 -> i32, add: i32 -> i32 -> i32`

Nicht LL(1)-konform, weil $\text{First}(\text{Type}) = \text{First}(\text{Primitive}) \cap \text{First}(\text{FuncType})$. $\text{First}(\text{Primitive}) = \{"i32", "f64"\}$. Und die Menge $\text{First}(\text{FuncType}) = \text{First}(\text{Primitive}) = \{"i32", "f64"\}$. Also der Durchschnitt der beiden Mengen $\neq \emptyset$. Man bräuchte 2 Lookaheads fürs korrekte Parsing.

Aufgabe 02 - Beseitigung von LL(1)-Konflikten mittels Faktorisierung

Beseitigen Sie in folgenden Grammatiken die enthaltenen LL(1)-Konflikte mittels Einsetzen und Faktorisierung. Geben Sie dazu eine umgeformte, ggf. vereinfachte EBNF Grammatik an.

a.)

```
StructDecl = ident "=" "{" { Value | NullableValue } "}" .  
Value = ident ":" Type .  
NullableValue = ident ":" "?" Type .  
Type = "i32" | "i64" | "string" .
```

Beispielsätze:

- `student = { name: string age: i32 uni: ?string }`
- `rect = { A: i64 B: i64 C: i64 D: ?i64 }`

Der Konflikt besteht hier bei den First-Mengen von `Value` und `NullableValue`. Der Durchschnitt der beiden Mengen ist nicht disjunkt, und deshalb ist die gegebene Grammatik nicht LL(1)-konform.

Man fixt den Konflikt, indem man in `StructDecl` für `Value` und `NullableValue` einsetzt, die gleichen Anfänge herauszieht (also `ident ":" Type`) und zuletzt noch das Fragezeichen als Option schreibt.

```
StructDecl = ident "=" "{" { ident ":" ["?"] Type } "}" .
```

b.)

```
Args = FixArgs ";" VarArgs .  
FixArgs = ident ":" Type { ";" ident ":" Type } .  
VarArgs = "... " ident .  
Type = "i32" | "string" .
```

Beispielsatz:

- `x: i32; n:string; p:i32; ...others`

Hier besteht ein hidden LL(1) Konflikt beim `;`. Der Parser kann nicht sagen, ob nach dem ersten `Type` von `FixArgs` als nächstes ein weiteres `FixArgs` kommt, oder doch ein `VarArgs`.

Der Konflikt kann so behoben werden, indem man den `;` von `FixArgs` "raus verschiebt" aus der Iteration (und natürlich auch am Ende noch eins hat, damit man den Beispielsatz erfüllen kann), und das Semikolon bei `Args` komplett entfernt. Somit sollte es keine Uneindeutigkeiten für `;` mehr geben.

```
Args = FixArgs VarArgs .  
FixArgs = ident ":" Type ";" {ident ":" Type ";"}
```

Aufgabe 03 - Beseitigung von Linksrekursion

Beseitigen Sie in folgender Grammatik die enthaltenen LL(1)-Konflikte. Geben Sie dazu eine umgeformte EBNF Grammatik ohne Rekursion an. Beseitigen Sie mögliche Linksrekursionen durch Umformung in eine Iteration (nicht durch Umformung in Rechtsrekursion).

```
Fields = Type Assigns .  
Assigns = ident "=" Expr .  
Expr = ident | number | Expr ( "==" | "!=" ) ( ident | number ) .  
Type = ident | Type "->" ident .
```

Beispielsätze:

- `int -> bool a = 5 == b`
- `bool b = 0`
- `int -> int -> int -> bool d = a == b != c`

```
Type = ident {"->" ident} .  
Expr = (ident | number) {"==" | "!="} (ident | number)} .
```

Aufgabe 04 - LL(4) Grammatik

Geben Sie eine einfache Grammatik mit mindestens 3 Produktionen an, welche die LL(4)-Eigenschaft, jedoch nicht die LL(3)-Eigenschaft besitzt.

$S = AbCX \mid AbCY.$

$A = a.$

$C = c.$

$X = x.$

$Y = y.$
